Backend · Proyecto final «Soluciones para el CLENA»

Arquitectura de CLENA Digital

CLENA Digital — Landing interactiva del CLENA

Documento de arquitectura · Entrega 1 · Versión 1.0 · 13 de noviembre de 2026

Curso de Backend, Universidad del Norte. Equipo de origen: Introducción a la Ingeniería de Sistemas (Samuel Lombana, Alan Fontalvo).

Integrante	Rol Scrum	Área técnica	GitHub
Samuel Ramos	Product Owner	Frontend, UX y accesibilidad	@sramoscor
Sebastián Brito	Scrum Master	DevOps y calidad	@sbritoa20
Camilo del Toro	Development Team (Tech Lead)	Arquitectura backend y seguridad	@CaDelTo
Jeison Acosta	Development Team	Base de datos y almacenamiento	@Aeon616

Repositorio: github.com/sbritoa20/Laboratorio-CLENA-Landing

Índice

1. Introducción 3
2. Contexto del sistema 3
3. Contenedores 4
4. Componentes de la API 6
    Capas 7
    Componentes del módulo de inscripciones 7
5. Código: inyección de dependencias 9
6. Flujo de una petición 11
7. Estructura de carpetas 13
8. Decisiones técnicas 15
9. Fragmento del módulo de NestJS 16
10. Requisitos transversales 17
11. Pendientes y riesgos 18

1. Introducción

CLENA Digital es una aplicación web con un frontend en React y una API REST en NestJS organizada con Clean Architecture sobre PostgreSQL. Este documento describe esa arquitectura en los niveles 1 a 3 del modelo C4, más el nivel 4 de código y el flujo de una petición.

Propósito y alcance. La landing presenta el CLENA (Corporación Luis Eduardo Nieto Arteta, antigua Aduana de Barranquilla): sus espacios, servicios, horarios, agenda de eventos y galería. El personal del CLENA administra ese contenido desde un panel, y cualquier visitante puede inscribirse a un evento y recibir la confirmación por correo.

Notación. Cada nivel de C4 se dibuja con el diagrama UML que le corresponde, para que todos los diagramas sigan el estándar UML:

Nivel C4	Diagrama UML	Figura	Qué muestra
1. Contexto	Componentes, con actores	Figura 1	El sistema como una caja, sus usuarios y los sistemas externos
2. Contenedores	Despliegue	Figura 2	Los procesos y almacenes que se ejecutan, y cómo se comunican
3. Componentes	Paquetes y componentes	Figuras 3 y 4	Las capas de la API y las piezas de un módulo
4. Código	Clases	Figura 5	Cómo un caso de uso recibe sus dependencias
Flujo de una petición	Secuencia	Figura 6	El recorrido de una inscripción por las capas

2. Contexto del sistema

CLENA Digital tiene cuatro tipos de usuario y depende de un solo sistema externo, el servidor de correo. La Figura 1 lo muestra en un diagrama de componentes UML: el sistema es un componente `«sistema»`, los usuarios son actores y cada relación dice qué se hace y por qué protocolo.

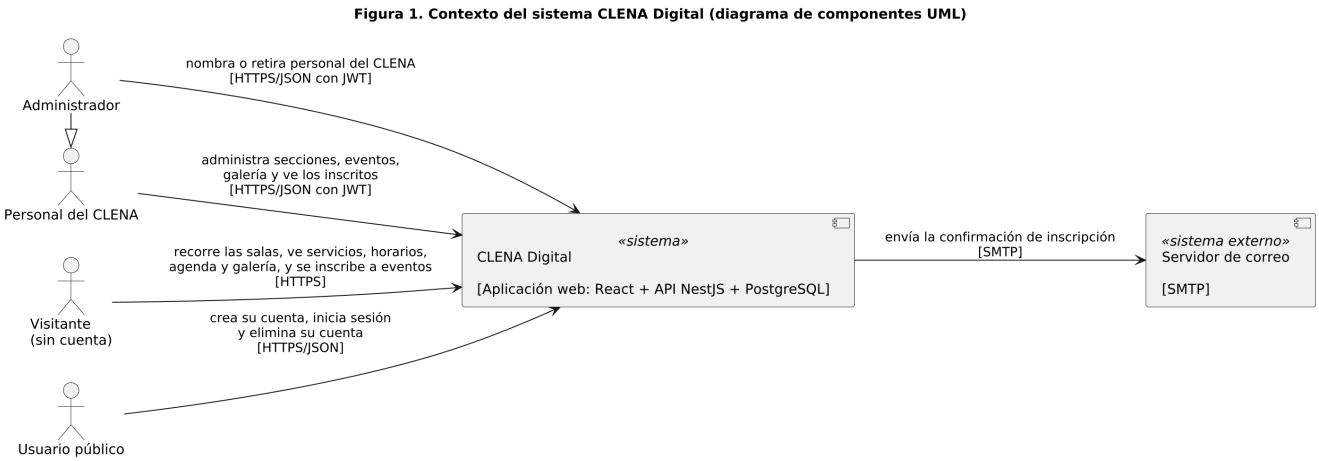


Figura 1. Contexto del sistema CLENA Digital (diagrama de componentes UML). Nivel 1 de C4.

Actor	Qué hace en el sistema	Cómo entra
Visitante (sin cuenta)	Recorre las salas, ve servicios, horarios, agenda y galería, y se inscribe a eventos	Sin sesión, por HTTPS
Usuario público	Crea su cuenta e inicia sesión	Registro e inicio de sesión
Personal del CLENA	Administra secciones, eventos y galería, y consulta los inscritos	Sesión con JWT, rol `staff`
Administrador	Todo lo del personal, más nombrar o retirar personal del CLENA	Sesión con JWT, rol `admin`

El administrador hereda del personal del CLENA (generalización en la figura). El único sistema externo es el servidor de correo, que recibe por SMTP la confirmación de cada inscripción.

3. Contenedores

El sistema corre en cuatro contenedores que `docker compose up` levanta sin pasos manuales: el frontend en Nginx, que sirve la aplicación y reenvía `/api`, `/archivos` y `/docs` a la API; la API en Node 22; PostgreSQL 16, y Mailpit, el servidor de correo de desarrollo. La API aplica las migraciones al arrancar. Dos volúmenes guardan los datos de PostgreSQL y las imágenes subidas, así que no se pierden al recrear los contenedores. Solo se publican el puerto 8080 (la aplicación) y el 8025 (la bandeja de Mailpit): la API y la base de datos no se alcanzan desde fuera de Docker.

Figura 2. Contenedores de CLENA Digital (diagrama de despliegue UML)

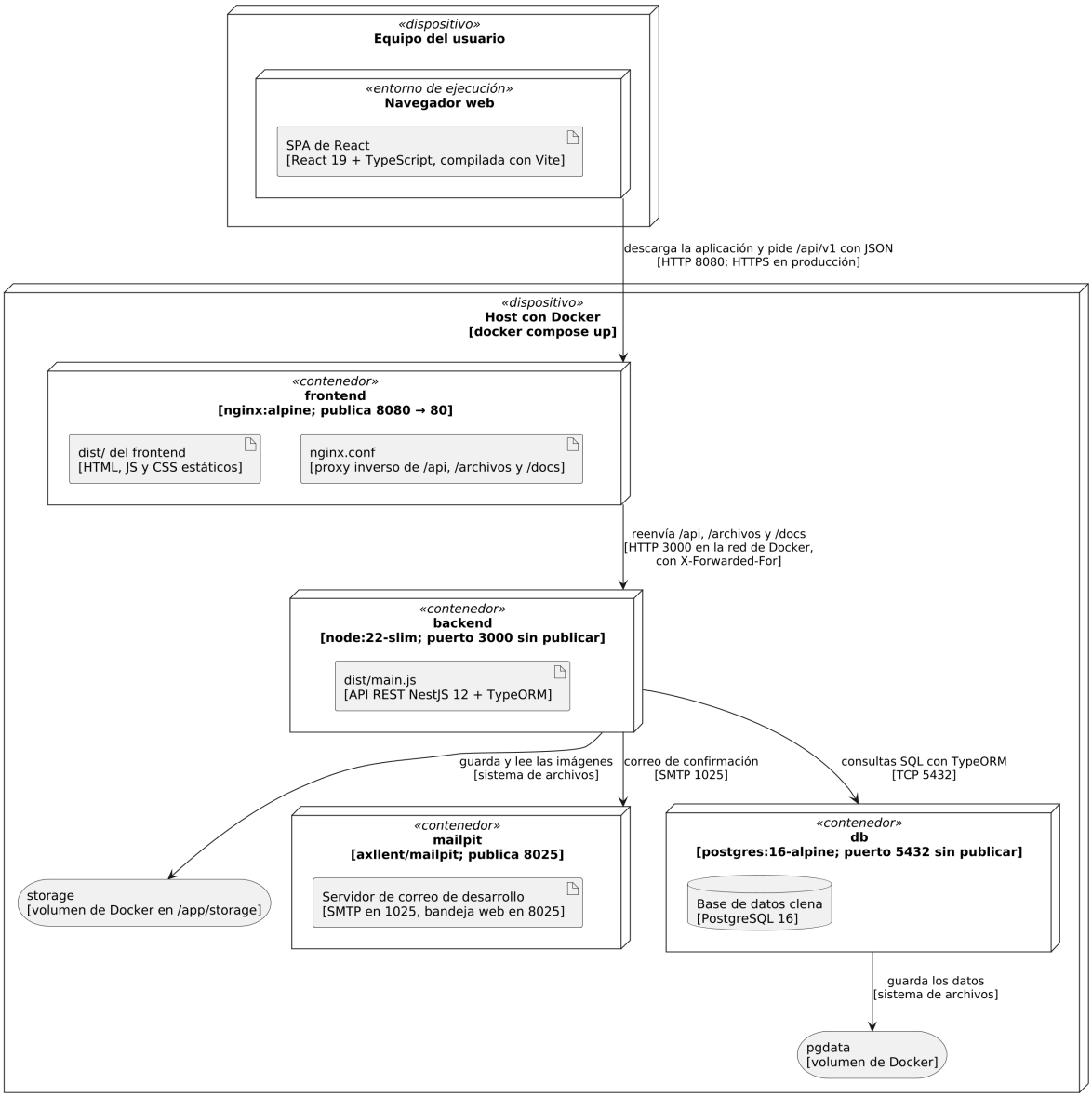


Figura 2. Contenedores de CLENA Digital (diagrama de despliegue UML). Nivel 2 de C4.

Contenedor	Tecnología	Responsabilidad	Puerto
`frontend`	React 19 + Vite, servido por Nginx	Entrega la aplicación web (SPA) al navegador y reenvía `/api`, `/archivos` y `/docs` a la API	8080 → 80
`backend`	NestJS 12 sobre Node 22	API REST en `/api/v1`, Swagger en `/docs`, migraciones al arrancar	3000, solo dentro de la red de Docker
`db`	PostgreSQL 16	Guarda usuarios, sesiones, secciones, eventos, inscripciones, imágenes de la galería y recorridos	5432, solo dentro de la red de Docker

Contenedor	Tecnología	Responsabilidad	Puerto
`mailpit`	Mailpit	Recibe por SMTP los correos de confirmación y los muestra en una bandeja web	1025 (SMTP, interno) y 8025 (web)
Volumen `pgdata`	Volumen de Docker	Conserva los datos de PostgreSQL	—
Volumen `storage`	Volumen de Docker en `/app/storage`	Conserva las imágenes de la galería y las portadas de los recorridos	—

Desde	Hacia	Qué viaja	Protocolo
Navegador	`frontend` (Nginx)	La aplicación (HTML, JS y CSS)	HTTP, puerto 8080
Navegador	`frontend` (Nginx)	Peticiones y respuestas JSON; access token en la cabecera y refresh token en una cookie httpOnly	HTTP/JSON, puerto 8080 (HTTPS en producción)
`frontend` (Nginx)	`backend`	Las peticiones a `/api`, `/archivos` y `/docs`, con la IP del visitante en `X-Forwarded-For`	HTTP 3000, dentro de la red de Docker
`backend`	`db`	Consultas SQL con TypeORM	TCP 5432
`backend`	`mailpit`	Correo de confirmación de cada inscripción	SMTP 1025
`backend`	Volumen `storage`	Las imágenes que sube el personal	Sistema de archivos
`db`	Volumen `pgdata`	Los datos de la base	Sistema de archivos

Requisito de despliegue. El frontend y la API deben quedar en el mismo sitio (mismo dominio), porque la cookie del refresh token es `SameSite=Strict`. En Docker se cumple porque el navegador solo habla con Nginx en localhost:8080, que reenvía /api a la API; en desarrollo, el proxy de Vite hace lo mismo.

4. Componentes de la API

La API sigue Clean Architecture: cuatro capas cuyas dependencias apuntan siempre hacia el dominio, y una carpeta `modules` que las une. Esa regla no depende de la disciplina del equipo: `npm run lint` falla en el CI si una capa importa algo que no le corresponde.

Capas

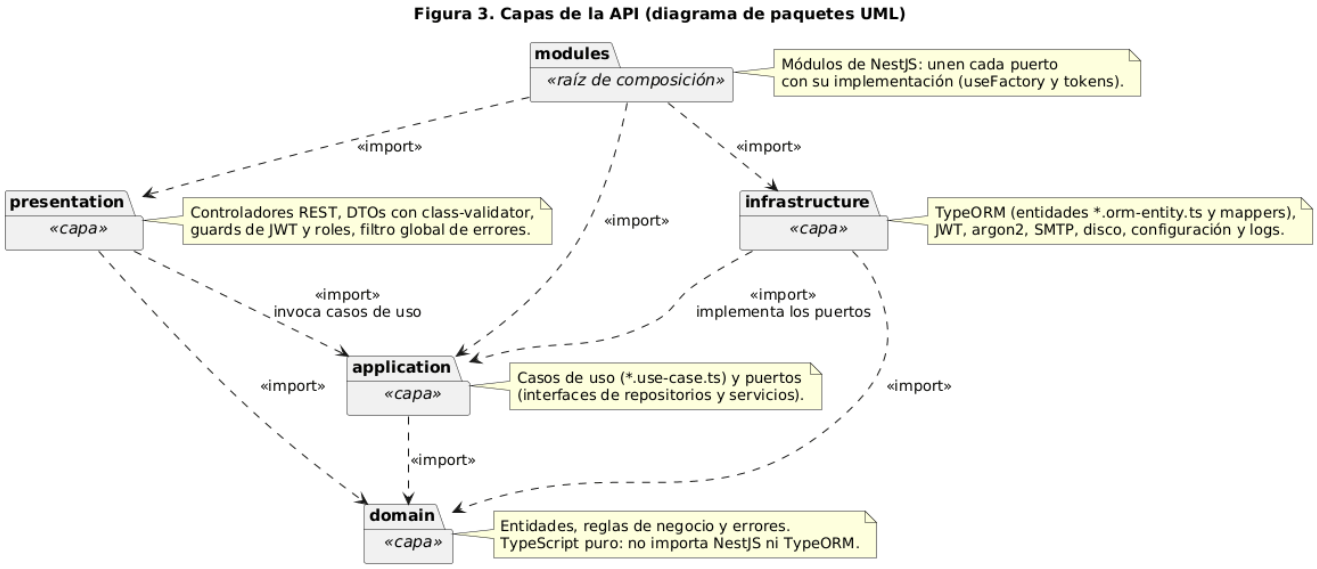


Figura 3. Capas de la API (diagrama de paquetes UML). Nivel 3 de C4.

Capa	Contiene	No puede importar
`domain`	Entidades, reglas de negocio y errores, en TypeScript puro	NestJS, TypeORM ni ninguna otra capa
`application`	Casos de uso y puertos (interfaces de repositorios y servicios)	Frameworks, `infrastructure`, `presentation` ni `modules`
`infrastructure`	Implementación de los puertos: TypeORM, JWT, argon2, SMTP, configuración y logs	`presentation` ni `modules`
`presentation`	Controladores REST, DTOs con validación, guards y filtro global de errores	`infrastructure`, `modules` ni TypeORM
`modules`	Módulos de NestJS que asocian cada puerto con su implementación	— (es la raíz de composición)

Componentes del módulo de inscripciones

El módulo de inscripciones se eligió como ejemplo porque tiene todos los tipos de pieza: controlador, dos casos de uso, tres puertos, sus implementaciones con TypeORM y SMTP, y dos entidades de dominio.

Figura 4. Componentes del módulo de inscripciones (diagrama de componentes UML)

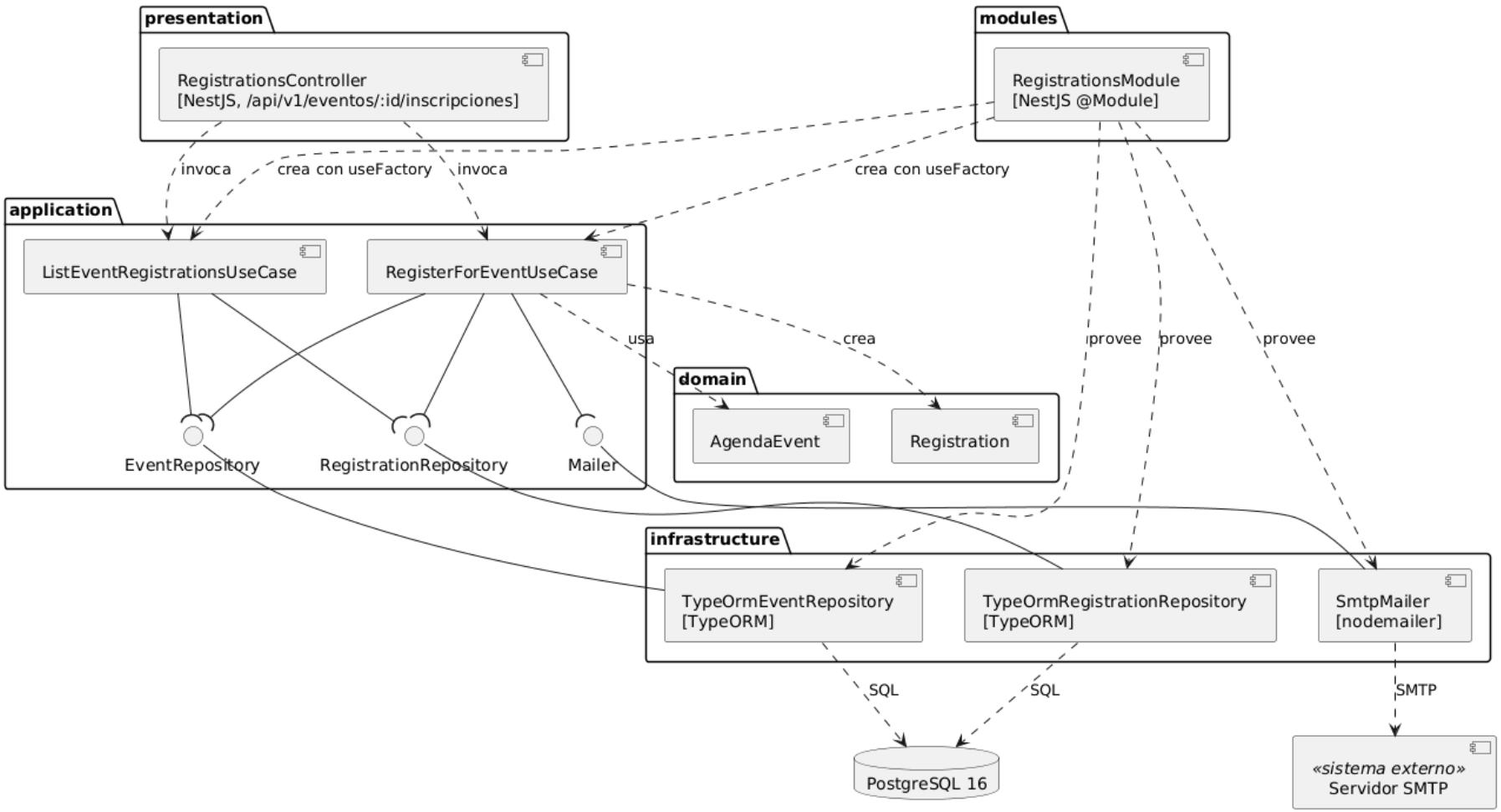


Figura 4. Componentes del módulo de inscripciones (diagrama de componentes UML). Nivel 3 de C4.

Los casos de uso **requieren** las interfaces `EventRepository`, `RegistrationRepository` y `Mailer`, y la infraestructura las **ofrece**. El caso de uso nunca sabe si detrás hay PostgreSQL o una versión en memoria, que es la que usan las pruebas unitarias.

Los demás módulos siguen el mismo patrón:

Módulo	Responsabilidad	Rutas
HealthModule	Estado de la API y de la base de datos	`GET /api/v1/health`
AuthModule	Registro, inicio de sesión, renovación y cierre de sesión	`/api/v1/auth/*`
AccountModule	Eliminar la propia cuenta y sus datos (Ley 1581)	`DELETE /api/v1/cuenta`
UsersModule	Listar usuarios y cambiar su rol (solo administrador)	`/api/v1/usuarios`
SectionsModule	Espacios, servicios y horarios de la landing	`/api/v1/secciones`
EventsModule	Agenda de eventos: la pública muestra solo lo publicado según sus fechas, y el personal programa, edita y elimina eventos	`/api/v1/eventos`
RegistrationsModule	Inscripción a eventos y lista de inscritos	`/api/v1/eventos/:id/inscripciones`
GalleryModule	Galería: subir, ver y eliminar imágenes	`/api/v1/galeria`
StorageModule	Guarda los archivos en disco y los sirve	`/archivos`
NotificationsModule	Envío de correos por SMTP	—
ToursModule	Recorridos por las salas (funcionalidad adicional)	`/api/v1/recorridos`

Cada módulo con base de datos tiene además un módulo `*-persistence` (por ejemplo, `EventsPersistenceModule`) que asocia el token de su repositorio con la implementación de TypeORM. Así otro módulo puede usar el repositorio sin cargar los controladores.

5. Código: inyección de dependencias

`RegisterForEventUseCase` recibe sus tres puertos por el constructor y no conoce ninguna implementación. Es `RegistrationsModule` el que decide qué clase concreta va en cada puerto, buscando cada una por su token de inyección.

Figura 5. Caso de uso que recibe sus puertos por token de inyección (diagrama de clases UML)

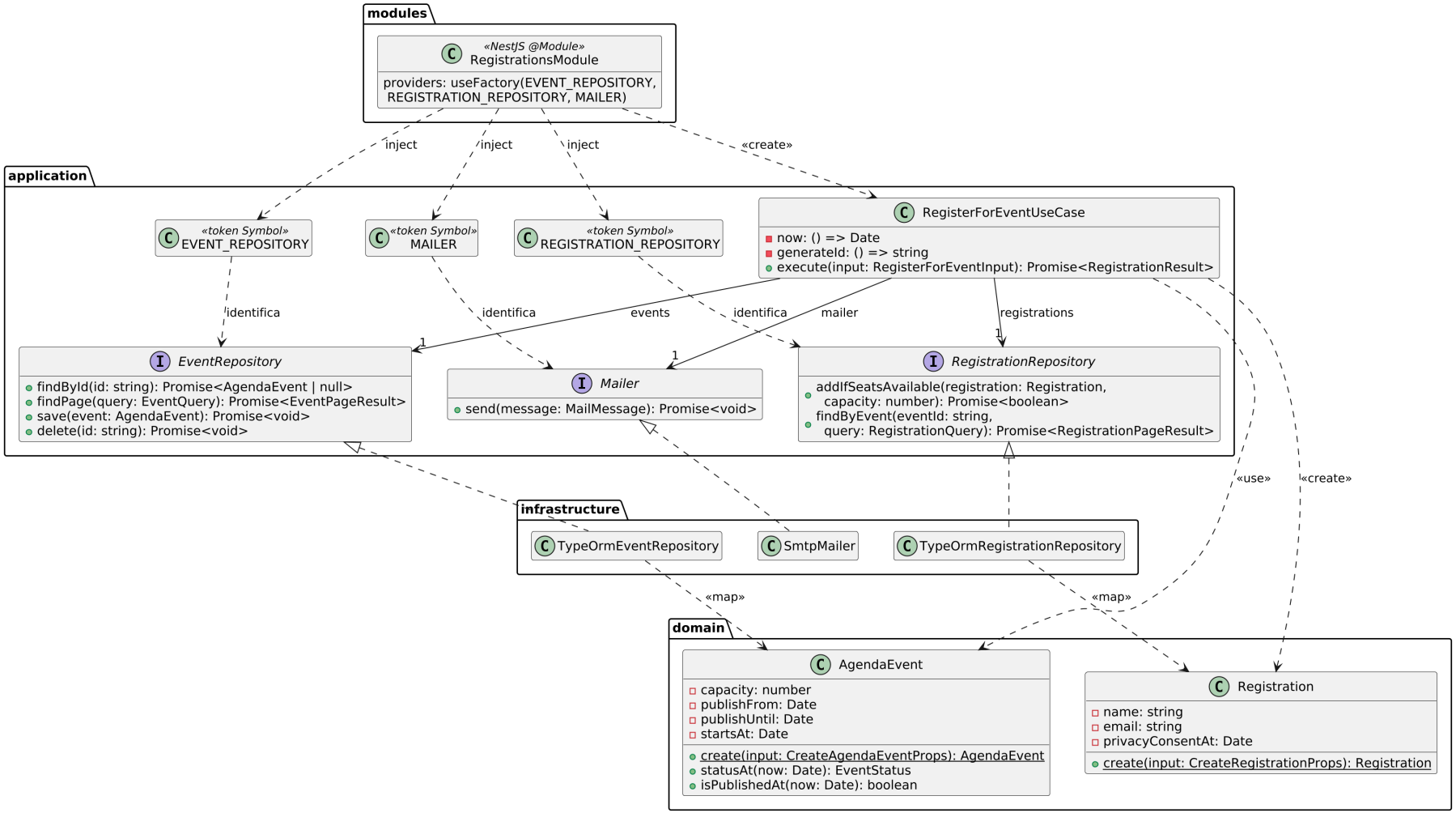


Figura 5. Caso de uso que recibe sus puertos por token de inyección (diagrama de clases UML). Nivel 4 de C4.

Cómo se lee, de arriba abajo:

1. `RegistrationsModule` crea el caso de uso con `useFactory` y le inyecta lo que encuentra bajo cada token.
2. Cada token es un `Symbol` (`EVENT_REPOSITORY`, `REGISTRATION_REPOSITORY`, `MAILER`) que identifica una interfaz. TypeScript borra las interfaces al compilar, así que NestJS necesita un valor real para encontrarlas.
3. El caso de uso solo depende de las interfaces de `application`.
4. `TypeOrmEventRepository`, `TypeOrmRegistrationRepository` y `SmtpMailer` implementan esas interfaces y convierten las filas de la base de datos en entidades de dominio.
5. `AgendaEvent` y `Registration` guardan las reglas de negocio: si el evento está publicado, si hay consentimiento, si el cupo es válido.

El reloj (`now`) y el generador de identificadores (`generateId`) también se inyectan, con valores por defecto. Así las pruebas unitarias fijan la fecha y el id sin trucos.

6. Flujo de una petición

Una inscripción pasa por cinco filtros antes de guardarse, y cada rechazo sale con un código de error distinto. La Figura 6 sigue `POST /api/v1/eventos/{id}/inscripciones` de principio a fin.

Figura 6. Inscribirse a un evento: POST /api/v1/eventos/{id}/inscripciones (diagrama de secuencia UML)

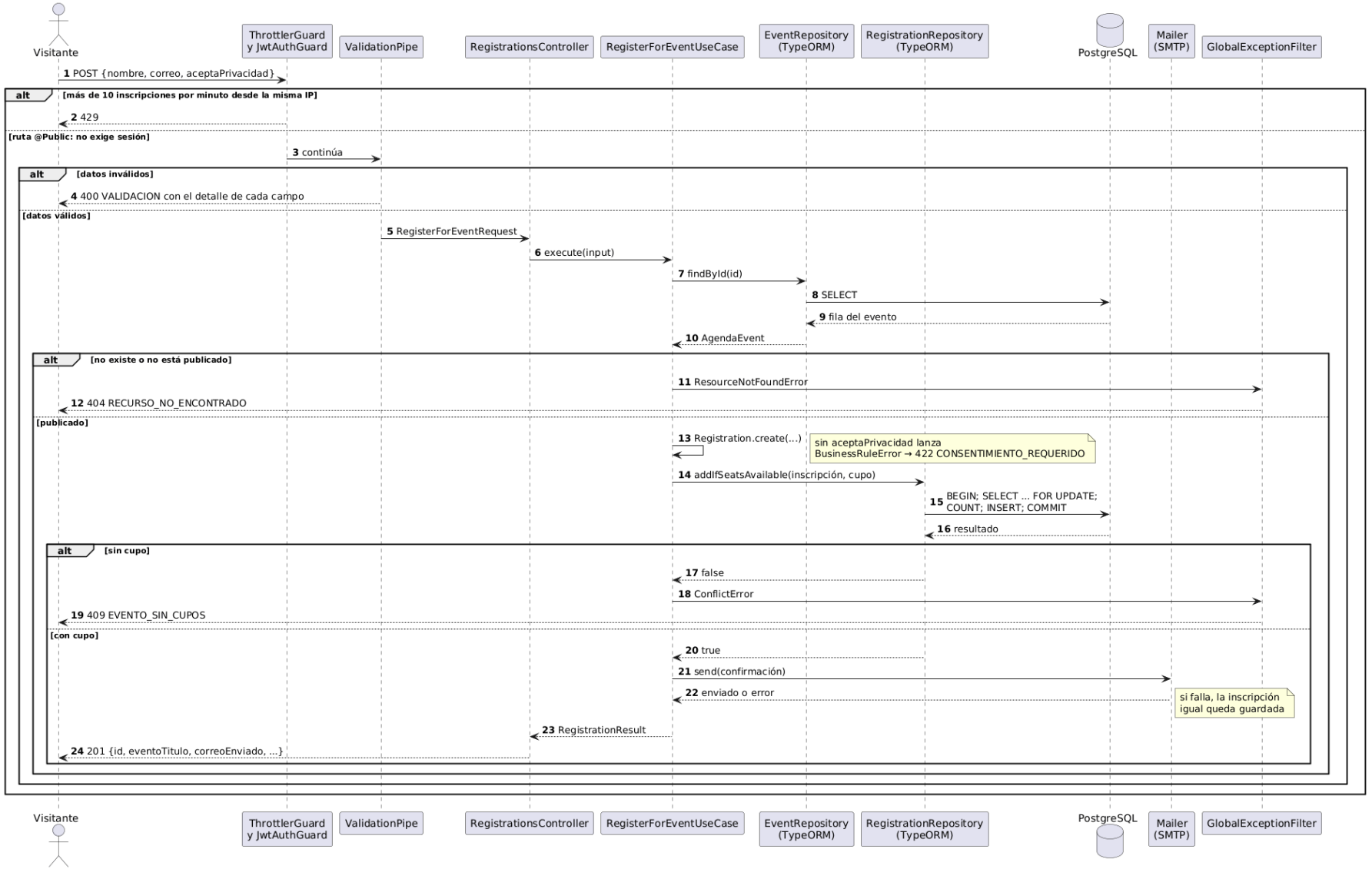


Figura 6. Inscribirse a un evento (diagrama de secuencia UML).

Paso	Quién	Si falla
Límite de intentos	`ThrottlerGuard`	429, más de 10 inscripciones por minuto desde la misma IP
Sesión	`JwtAuthGuard` (global)	No aplica: la ruta es `@Public()`
Datos de entrada	`ValidationPipe` (global)	400 `VALIDACION`, con el error de cada campo
Evento publicado	`RegisterForEventUseCase` y `AgendaEvent`	404 `RECURSO_NO_ENCONTRADO`
Consentimiento	`Registration.create`	422 `CONSENTIMIENTO_REQUERIDO`
Cupo	`RegistrationRepository`	409 `EVENTO_SIN_CUPOS`

Cupo sin sobreventa. El cupo se revisa y la inscripción se guarda dentro de una misma transacción que bloquea la fila del evento (`SELECT … FOR UPDATE`). Si dos personas se inscriben a la vez al último cupo, la segunda espera a que la primera termine y recibe 409.

El correo no bloquea la inscripción. Si el servidor de correo falla, la inscripción igual queda guardada y la respuesta trae `correoEnviado: false`.

Errores uniformes. Los casos de uso lanzan errores de dominio y el `GlobalExceptionFilter` los traduce a HTTP con la misma forma JSON (sección 10).

7. Estructura de carpetas

Cada capa es una carpeta de `backend/src`, y dentro de cada capa hay una subcarpeta por funcionalidad. Una funcionalidad completa, como las inscripciones, queda repartida así:

```
backend/
├── src/
│   ├── domain/
│   │   ├── errors/                    Errores de dominio (404, 403, 409, 422)
│   │   └── registrations/
│   │       └── registration.ts        Entidad y sus reglas
│   ├── application/
│   │   ├── common/pagination.ts       Paginación compartida
│   │   └── registrations/
│   │       ├── ports/                 Interfaces y sus tokens
│   │       └── register-for-event.use-case.ts
│   ├── infrastructure/
│   │   ├── config/  logging/          Variables de entorno y logs
│   │   ├── database/migrations/       Migraciones versionadas
│   │   └── registrations/
│   │       ├── registration.orm-entity.ts
│   │       ├── registration.mapper.ts
│   │       └── typeorm-registration.repository.ts
│   ├── presentation/http/
│   │   ├── auth/  errors/  validation/ Guards, filtro de errores y validación
│   │   └── registrations/
│   │       ├── registrations.controller.ts
│   │       └── registrations.dto.ts
│   ├── modules/
│   │   └── registrations.module.ts    Une puertos e implementaciones
│   ├── app.module.ts
│   └── main.ts
└── test/
    ├── *.e2e-spec.ts                  Pruebas de la API completa
    └── support/                       Repositorios en memoria para las pruebas
```

Tipo de archivo	Capa	Ejemplo
Entidad de dominio	`domain/<funcionalidad>/`	`registration.ts`
Caso de uso	`application/<funcionalidad>/`	`register-for-event.use-case.ts`
Puerto (interfaz + token)	`application/<funcionalidad>/ports/`	`registration-repository.ts`
Entidad del ORM	`infrastructure/<funcionalidad>/`	`registration.orm-entity.ts`
Mapper ORM ↔ dominio	`infrastructure/<funcionalidad>/`	`registration.mapper.ts`
Controlador y DTOs	`presentation/http/<funcionalidad>/`	`registrations.controller.ts`
Módulo de NestJS	`modules/`	`registrations.module.ts`
Prueba unitaria	Junto al archivo que prueba	`registration.spec.ts`

Convenciones. Clases, archivos, funciones y variables van en inglés. Los campos del JSON, los códigos de error y los mensajes para el usuario van en español, porque son lo que ve el frontend y el usuario final.

Frontend. `frontend/src` se organiza en `api/` (cliente de la API), `componentes/`, `paginas/`, `sesion/` (sesión y rutas protegidas), `datos/` y `tipos/`.

8. Decisiones técnicas

Cada decisión se tomó frente a una alternativa concreta; la tabla dice cuál y por qué se descartó.

Tema	Decisión	Alternativa descartada	Por qué
Framework	NestJS 12 con TypeScript	Express sin estructura	Inyección de dependencias, módulos y guards incluidos; encaja con Clean Architecture
ORM	TypeORM 1.1 con migraciones versionadas y `synchronize` apagado	Prisma	Se integra con NestJS sin un generador aparte; las migraciones quedan en el repositorio y el esquema nunca cambia solo
Base de datos	PostgreSQL 16	MySQL	Transacciones con `SELECT … FOR UPDATE` para el cupo de los eventos
Validación	class-validator y class-transformer en un `ValidationPipe` global	Validar a mano en cada controlador	Los DTOs declaran las reglas una sola vez y Swagger las documenta
Logger	pino con nestjs-pino	Logger por defecto de NestJS	Logs en JSON y ocultación de datos sensibles (la cabecera `Authorization` y las cookies del refresh token)
Autenticación	JWT de 15 minutos más refresh token opaco de 7 días, con rotación	Sesión en el servidor o JWT de larga duración	Un token robado sirve poco tiempo; reusar un refresh token ya rotado cierra todas las sesiones del usuario
Contraseñas	argon2id (`@node-rs/argon2`)	bcrypt	Recomendado por OWASP; resiste mejor ataques con GPU
Documentación de la API	Swagger (`@nestjs/swagger`) en `/docs`	Documentación escrita a mano	Se genera desde los DTOs y no se desactualiza
Pruebas	Vitest, unitarias y e2e	Jest	Más rápido y compatible con ESM, que usa el proyecto
Reglas de capas	oxlint con reglas de importación	Revisión manual	El CI falla si una capa importa lo que no debe
Correo	nodemailer por SMTP; Mailpit en desarrollo	Servicio de correo en la nube	Sin cuentas ni costos; Mailpit muestra los correos en local
Integración continua	GitHub Actions	—	Lint, pruebas unitarias con cobertura mínima de 70 % en dominio y aplicación, pruebas e2e y build en cada PR; `main` exige el CI en verde y una aprobación
Frontend	React 19, Vite 7 y React Router 7	—	Exigido por el curso
Librería de componentes	Componentes propios con CSS, y Photo Sphere Viewer para el visor 360°	Material UI	La landing tiene un diseño propio; una librería genérica agregaba peso sin ahorrar trabajo

9. Fragmento del módulo de NestJS

`RegistrationsModule` es el único lugar que sabe qué implementación corresponde a cada puerto. Importa los módulos que proveen los repositorios y el correo, y crea cada caso de uso con `useFactory`, inyectando lo que encuentra bajo cada token.

```
/** Inscripciones a eventos (#46). */
@Module({
  imports: [
    EventsPersistenceModule,        // provee EVENT_REPOSITORY
    RegistrationsPersistenceModule, // provee REGISTRATION_REPOSITORY
    NotificationsModule,            // provee MAILER
  ],
  controllers: [RegistrationsController],
  providers: [
    {
      provide: RegisterForEventUseCase,
      inject: [EVENT_REPOSITORY, REGISTRATION_REPOSITORY, MAILER],
      useFactory: (
        events: EventRepository,
        registrations: RegistrationRepository,
        mailer: Mailer,
      ) => new RegisterForEventUseCase(events, registrations, mailer),
    },
    {
      provide: ListEventRegistrationsUseCase,
      inject: [EVENT_REPOSITORY, REGISTRATION_REPOSITORY],
      useFactory: (
        events: EventRepository,
        registrations: RegistrationRepository,
      ) => new ListEventRegistrationsUseCase(events, registrations),
    },
  ],
})
export class RegistrationsModule {}
```

El módulo de persistencia asocia el token con su clase concreta:

```
// registrations-persistence.module.ts
{
  provide: REGISTRATION_REPOSITORY,
  inject: [getRepositoryToken(RegistrationOrmEntity)],
  useFactory: (repository: Repository<RegistrationOrmEntity>) =>
    new TypeOrmRegistrationRepository(repository),
}
```

El caso de uso no lleva decoradores de NestJS: es una clase de TypeScript puro. Por eso se puede probar con repositorios en memoria y no depende del framework.

10. Requisitos transversales

Seguridad, errores y privacidad se resuelven una sola vez para toda la API, no en cada endpoint.

Roles y rutas protegidas. Toda ruta exige un access token válido salvo las marcadas con `@Public()`. Dos guards globales hacen cumplir la regla: `JwtAuthGuard` revisa el token y `RolesGuard` el rol que pide `@Roles(...)`.

Rol	Código	Puede
Administrador	`admin`	Todo, más listar usuarios y cambiar su rol
Personal del CLENA	`staff`	Administrar secciones, eventos y galería, y ver los inscritos
Usuario público	`public`	Su perfil; es el rol de toda cuenta nueva

Sin token, o con uno vencido, la API responde 401; con un rol que no alcanza, 403. El primer administrador se crea con `npm run usuarios:hacer-admin -- <correo>`.

Formato uniforme de errores. El `GlobalExceptionFilter` responde todos los errores con la misma forma:

```
{
  "estado": 400,
  "codigo": "VALIDACION",
  "mensaje": "Los datos enviados no son válidos",
  "detalles": [{ "campo": "correo", "errores": ["El correo no es válido"] }],
  "ruta": "/api/v1/eventos/1/inscripciones",
  "fecha": "2026-11-13T09:00:00.000Z"
}
```

Error de dominio	HTTP
`ResourceNotFoundError`	404
`AccessDeniedError`	403
`ConflictError`	409
`BusinessRuleError`	422
Cualquier otro	500, sin detalles internos; queda en los logs

Ley 1581 de 2012 (protección de datos).

- El registro y la inscripción exigen `aceptaPrivacidad: true`. Se guarda la fecha del consentimiento y la versión del aviso aceptado.
- La inscripción pide solo nombre y correo, los datos mínimos para confirmar el cupo.
- Cualquier usuario puede eliminar su cuenta desde Mi cuenta (`DELETE /api/v1/cuenta`, con su contraseña): se borran su nombre, correo y contraseña, sus sesiones, sus recorridos y sus inscripciones. Las fotos que subió a la galería se quedan sin su nombre, porque son contenido del CLENA.

- Las contraseñas se guardan con argon2id, y de los refresh tokens solo se guarda su SHA-256.
- Los logs ocultan la cabecera `Authorization` y las cookies.

Límite de intentos. Registro, inicio de sesión e inscripción aceptan 10 intentos por minuto por visitante; detrás de Nginx se cuenta la IP que agrega el proxy, no la que envía el cliente; después responden 429.

CORS. Solo los orígenes de `CORS_ORIGIN` pueden llamar a la API, con `credentials: true` para que viaje la cookie del refresh token.

11. Pendientes y riesgos

Tema	Estado	Qué falta
Despliegue, HTTPS y correo	La aplicación todavía no está publicada, y en Docker los correos llegan a Mailpit, que no los entrega a nadie	Publicarla con HTTPS en un solo dominio y configurar un servidor SMTP real con `SMTP_HOST` y `SMTP_PORT`; en producción la cookie del refresh token ya se marca `Secure`
Imágenes de ejemplo de la galería	Se cargan con `npm run galeria:semilla` dentro del contenedor	Correrlo antes de la demo, o agregarlo al arranque del compose
Primer administrador	Se crea con `npm run usuarios:hacer-admin` dentro del contenedor	Nada: está documentado en el README
Aviso de privacidad	Versión provisional (`2026-11-provisional`)	El texto definitivo, validado por el CLENA, y su versión
Límite de intentos	10 por minuto por IP en registro, inicio de sesión e inscripción	Un grupo que comparta red, como un colegio, puede alcanzarlo; revisar el límite antes de abrir inscripciones masivas